

CSC 406 - Banking Systems

BankApp User Manual

Version 1.3 · Spring 2026

Audience: Customers, Tellers, Bank Managers, System Controllers

System: CSC 406 Banking System Application – prototypeBranch

Covers: All user-facing screens: Customer, Teller, Manager, System Controller

Status: Draft – Updated to reflect prototypeBranch implementation

1. Introduction

This manual covers all screens of the CSC 406 Banking System application. It is organized by user role: Customer, Teller, Bank Manager, and System Controller. Each section documents every screen the role can access, describes every UI element and button, and explains the business rules that govern each action.

1.1 User Roles

Role	Responsibilities
Customer	Manages personal accounts, submits checks, makes credit card payments, and reviews account status.
Teller	Creates customers and accounts, links accounts, processes withdrawals and stop payments, and reviews customer accounts.
Bank Manager	Sets interest rates on loans and deposit accounts, establishes CDs, and processes checks.
System Controller	Loads initial data, runs regression tests, and manages system checkpoints and disaster recovery.

1.2 Screen Inventory

Role	Screens
Customer	Credit Card Screen, Review Account Status, Input Check Screen
Teller	Create New Customer, Create New Account, Link Accounts, Deposit to Account, Withdraw from Account, Stop Payment, Review Customer Accounts
Bank Manager	Create New Loan, Set Interest Rates on Loans, Set Interest Rates on CDs / Checking, Establish CDs, Process Checks
System Controller	Load Basic Data, Run Simple Regression Tests, Check Point System

1.3 Common Conventions

- Fields labeled in orange are required.
- Fields labeled in blue are optional or display-only.
- Every screen has a Return button that navigates back to the caller's home screen without saving.
- The system displays a confirmation message after every successful operation.
- Account numbers are generated automatically as UUIDs — tellers and managers do not need to assign them manually unless overriding.

2. Customer Screens

The following screens are accessible to authenticated Customers. Customers interact with the system to manage their own accounts, make payments, and submit checks. The Customer home screen provides navigation to each of the three activity screens below.

2.1 Credit Card Screen

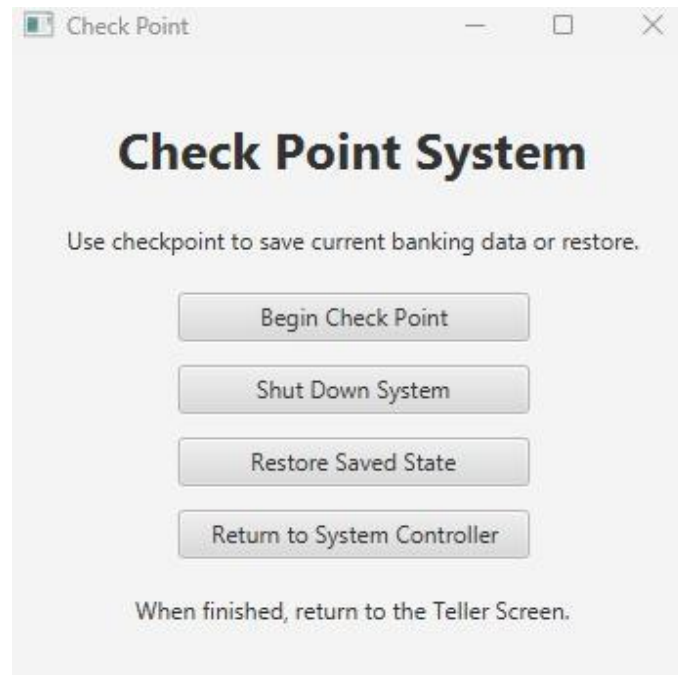


Figure 2.1 – Credit Card Screen

This screen allows a Customer to make a payment toward their Credit Card account balance. The system retrieves and displays the current balance before the customer enters a payment amount.

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the unique Customer ID associated with the account. Used to look up the customer record.	<i>Required. Must match an existing customer in the system.</i>
Credit Card Account Number	Text field	Enter the credit card account number to apply the payment to.	<i>Required. Must be a valid CreditCard account owned by the entered Customer ID.</i>
Current Balance	Display field	Read-only field showing the current outstanding balance on the credit card once the account is located. Not editable.	<i>Auto-populated. Reflects currentPaymentDue from CreditCard object.</i>

UI Element	Type	What It Does	Notes / Validation
Payment Amount	Text field	Enter the dollar amount the customer wishes to pay toward the credit card balance.	<i>Required. Must be greater than \$0. CreditCard.makePayment() clears financeCharge first, then applies to principal.</i>
Make Payment	Button	Submits the payment. Deducts the entered amount from the credit card balance, logs a Transaction entry, and updates averageDue. Returns to Customer Screen on success.	<i>Calls CreditCard.makePayment(amount). Finance charge is cleared before principal.</i>
Return to Customer Screen	Button	Cancels the operation and returns to the Customer home screen without making any changes.	<i>No data is saved when this button is used.</i>

2.2 Review Account Status

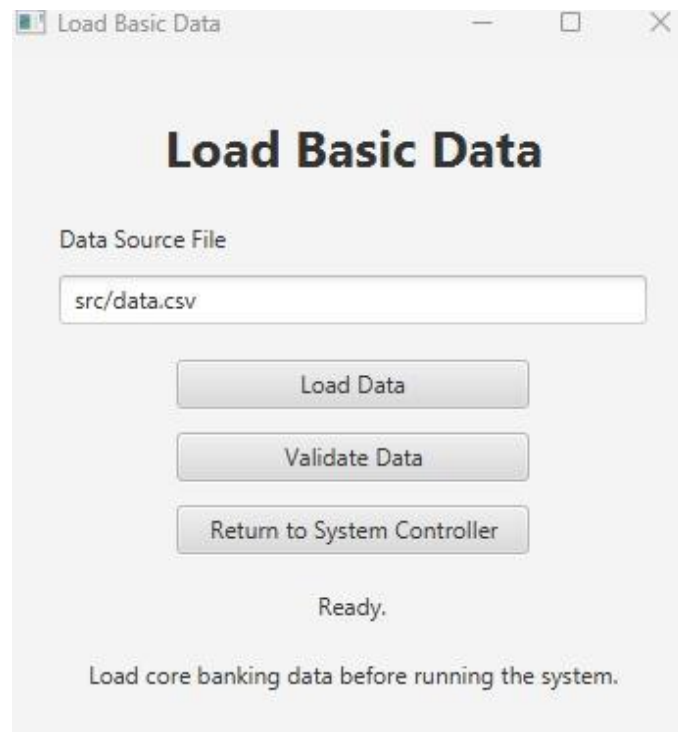


Figure 2.2 – Review Account Status

This screen provides a read-only overview of all accounts associated with the logged-in customer. Account details are displayed in a summary table that refreshes on demand.

UI Element	Type	What It Does	Notes / Validation
Customer Account Overview	Section header	Identifies the section displaying account data. Not interactive.	<i>Display only.</i>

UI Element	Type	What It Does	Notes / Validation
Account Number	Table column	Displays the UUID account number for each account owned by the customer.	<i>Read-only. Generated by <code>UUID.randomUUID()</code> at account creation.</i>
Type	Table column	Displays the account type: Savings, TMB Checking, GD Checking, CD, or Credit Card.	<i>Read-only.</i>
Balance	Table column	Displays the current balance from <code>getBalance()</code> for each account.	<i>Read-only. Formatted as currency.</i>
Status	Table column	Displays the account standing: Active, Frozen, Overdue, etc. CreditCard shows <code>isProblemAccount</code> flag.	<i>Read-only.</i>
Refresh Accounts	Button	Reloads the account data from the system and repopulates the table with the latest balances and statuses.	<i>Use after deposits, withdrawals, or payments to see updated figures.</i>
Return to Customer Screen	Button	Navigates back to the Customer home screen.	<i>No data changes are made by this screen.</i>

2.3 Input Check Screen



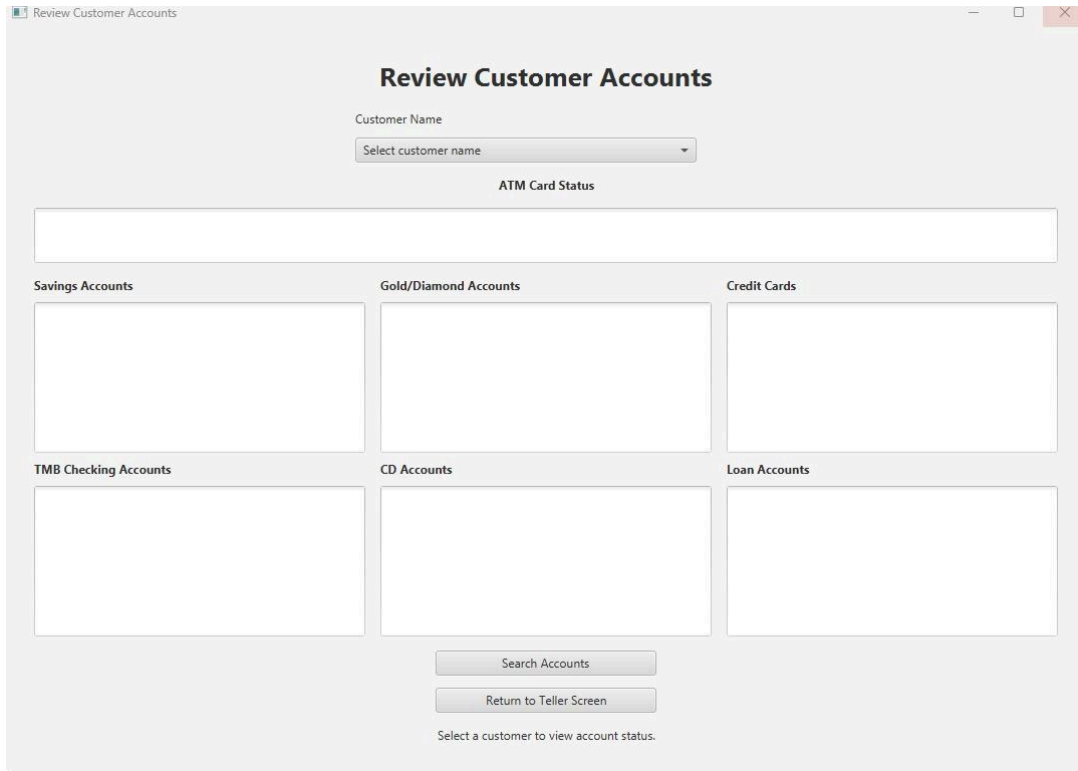
Figure 2.3 – Input Check Screen

This screen allows a Customer to submit a check for processing against their checking account. The system validates the check details and processes it, applying overdraft fee-waiver logic via requestFeeWaiver() if the account balance is sufficient (Sequence Diagram Scenario H).

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the Customer ID of the account holder presenting the check.	<i>Required. Must match an existing customer.</i>
Checking Account Number	Text field	Enter the checking account number that the check will be drawn from.	<i>Required. Must be a valid TMBAccount or GDAccount for the customer.</i>
Check Number	Text field	Enter the check number as printed on the physical check.	<i>Required. Used for stop-payment lookup and audit trail.</i>
Check Amount	Text field	Enter the dollar amount written on the check.	<i>Required. Must be greater than \$0.</i>
Submit Check	Button	Processes the check. Calls CheckingsAccount.requestFeeWaiver(checkAmt) — returns true if (balance - checkAmt) >= \$5,000, waiving the \$25 fee. Otherwise overdraft handling applies via handleOverdraft().	<i>Fee waiver is a local balance check in this build. Full CheckpointManager integration is planned.</i>
Return to Customer Screen	Button	Cancels check submission and returns to the Customer home screen.	<i>No data is saved.</i>

3. Teller Screens

Tellers have access to seven activity screens for account and customer administration. These screens are accessed from the Teller home screen shown below.



Review Customer Accounts

Customer Name
Select customer name

ATM Card Status

Savings Accounts

Gold/Diamond Accounts

Credit Cards

TMB Checking Accounts

CD Accounts

Loan Accounts

Search Accounts

Return to Teller Screen

Select a customer to view account status.

Teller Screen – Home

3.1 Create New Customer

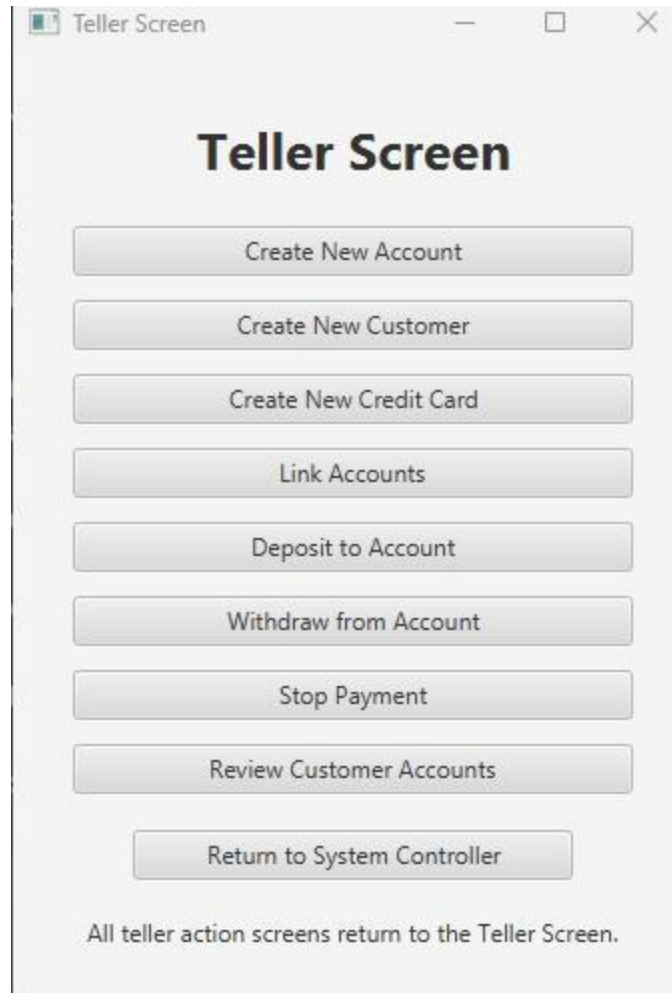


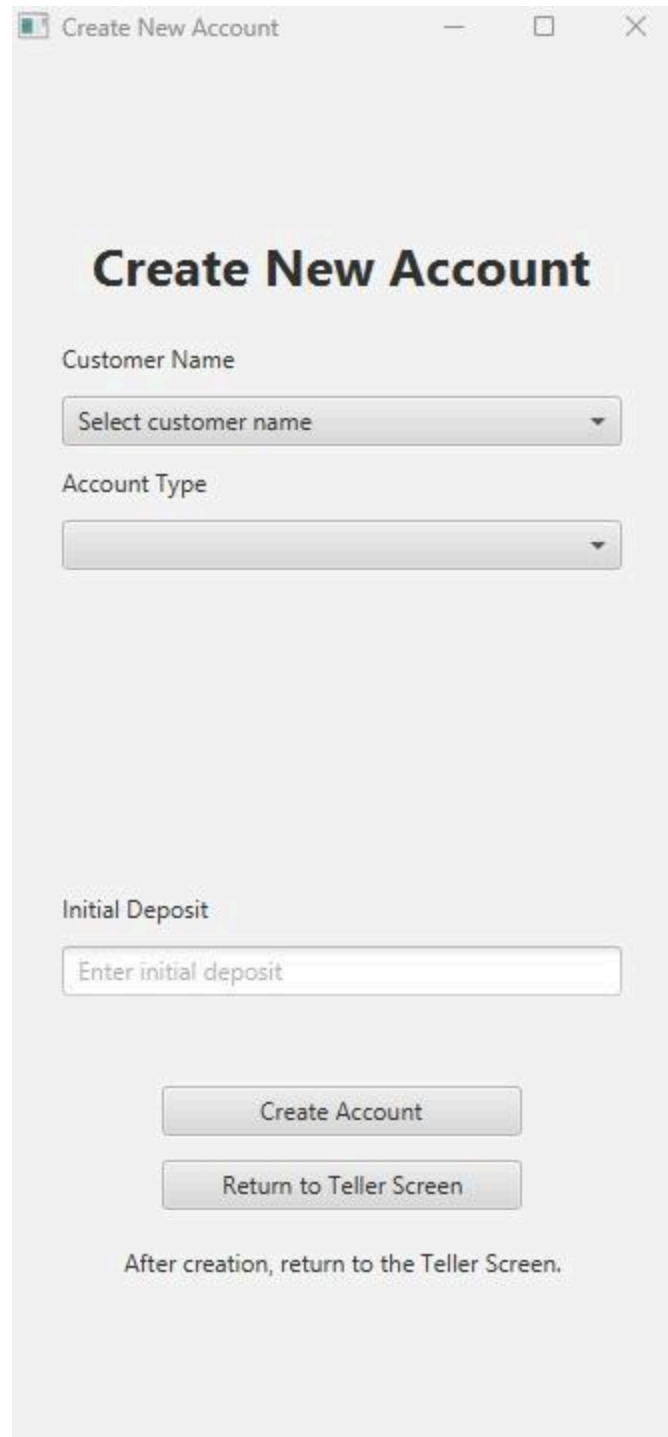
Figure 3.1 – Create New Customer

This screen allows a Teller to register a new Customer in the system. The field order on screen is: Customer ID, Address, City, State, Zipcode, First Name, Last Name. Once created the customer is inserted into the sorted customer list and becomes available for account creation.

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the unique identifier to assign to the new customer (used as SSN/ID). This is the primary lookup key.	<i>Required. Must be unique in the system.</i>
Address	Text field	Enter the customer's street address.	<i>Required. Stored in Customer.address.</i>
City	Text field	Enter the customer's city.	<i>Required. Stored in Customer.city.</i>
State	Text field	Enter the 2-letter state code (e.g. MO, KS).	<i>Required. Stored in Customer.state.</i>

UI Element	Type	What It Does	Notes / Validation
Zipcode	Text field	Enter the 5-digit ZIP code.	<i>Required. Stored in Customer.zipcode.</i>
First Name	Text field	Enter the customer's legal first name.	<i>Required. Stored in Customer.firstName.</i>
Last Name	Text field	Enter the customer's legal last name. The system sorts customers alphabetically by last name.	<i>Required. Stored in Customer.lastName.</i>
Create Customer	Button	Submits the new customer record. Creates the Customer object and inserts it in sorted order via ArrayListManager.addInOrder(). Status label shows 'Ready.' on success.	<i>Duplicate IDs are rejected. Status updates on completion.</i>
Return to Teller Screen	Button	Cancels customer creation and returns to the Teller home screen with no changes.	<i>No data is saved.</i>

3.2 Create New Account



The image shows a software window titled "Create New Account". Inside the window, the title "Create New Account" is displayed in a large, bold font. Below the title, there are three main input sections. The first section is labeled "Customer Name" and contains a dropdown menu with the text "Select customer name" and a downward arrow. The second section is labeled "Account Type" and contains an empty dropdown menu with a downward arrow. The third section is labeled "Initial Deposit" and contains a text input field with the placeholder text "Enter initial deposit". Below these input fields, there are two buttons: "Create Account" and "Return to Teller Screen". At the bottom of the window, there is a line of text that reads "After creation, return to the Teller Screen."

Figure 3.2 – Create New Account

This screen allows a Teller to open a new bank account for an existing customer. The customer is selected from a dropdown populated from `AppState.customers`. Account type is selected from a second dropdown.

UI Element	Type	What It Does	Notes / Validation
Customer Name	Dropdown	Select the customer to open the account for from the list of all loaded customers.	<i>Required. Populated from AppState.customers at screen load.</i>
Account Type	Dropdown	Select the account type: Savings Account, Gold Diamond Checking Account, That's My Bank Checking Account, or CD Account.	<i>Required. Account number is auto-generated as a UUID. Type determines applicable fees and minimum balance rules.</i>
Initial Deposit	Text field	Enter the opening deposit amount for the new account.	<i>Required. GDAccount minimum is \$5,000. TMBAccount and Savings accept any positive amount.</i>
Create Account	Button	Creates the new account with the selected type and deposit. Links it to the selected customer's accountList. Returns to Teller Screen on success.	<i>System validates minimum balance before creation. Status shown at bottom of screen.</i>
Return to Teller Screen	Button	Cancels account creation and returns to the Teller home screen.	<i>No data is saved.</i>

3.3 Link Accounts

The screenshot shows a software window titled "Create New Credit Card". Inside the window, the title "Create New Credit Card" is prominently displayed at the top. Below the title, there are four input fields arranged vertically: a dropdown menu labeled "Select Customer", and three text input fields labeled "Credit Limit", "Interest Rate", and "Starting Balance / Amount Due". At the bottom of the form, there are two buttons: "Create Credit Card" and "Return to Teller Screen".

Figure 3.3 – Link Accounts

This screen allows a Teller to establish a relationship between two of a customer's accounts. All fields use dropdowns populated from the live customer and account data. Linking a SavingsAccount to a CheckingsAccount enables automatic overdraft protection.

UI Element	Type	What It Does	Notes / Validation
Customer name	Dropdown	Select the customer whose accounts are to be linked.	<i>Required. Populated from AppState.customers.</i>
Primary Account	Dropdown	Select the primary account (the TMBAccount or GDAccount to receive overdraft protection).	<i>Required. Populated from selected customer's accountList.</i>
Linked Account	Dropdown	Select the SavingsAccount to link as the overdraft backup.	<i>Required. Must be a different account from the primary.</i>
Link Type	Dropdown	Select the relationship type (e.g. Overdraft Protection).	<i>Required. Overdraft Protection calls setOverdraftProtAccount() on the CheckingsAccount.</i>
Link Accounts	Button	Creates the link. Calls CheckingsAccount.setOverdraftProtAccount(savingsAccount), which also calls toggleOverdraftBackup() on the savings account.	<i>Once linked, handleOverdraft() auto-transfers from savings when checking goes negative, at no fee.</i>
Return to Teller Screen	Button	Cancels the operation and returns to the Teller home screen.	<i>No data is saved.</i>

3.4 Deposit to Account

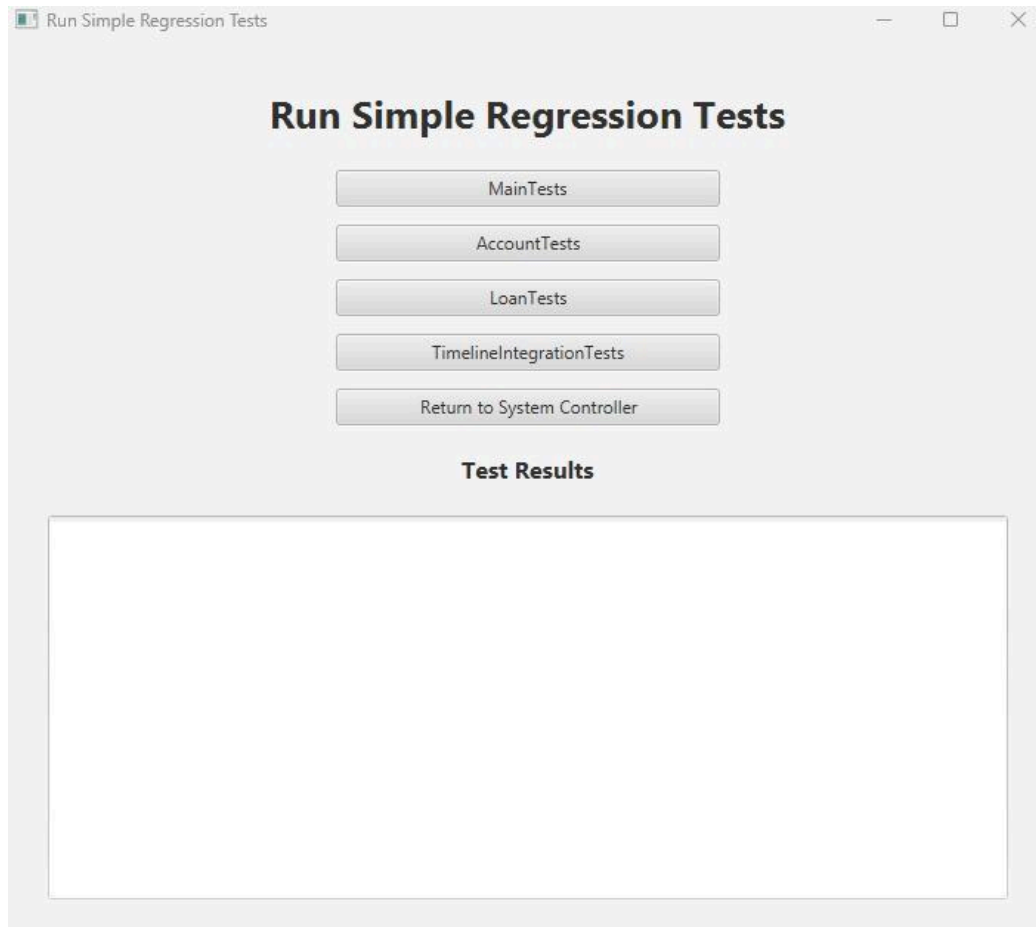


Figure 3.4 – Deposit to Account

This is a new screen that allows a Teller to deposit funds into a customer's account. After a successful deposit, the screen shows a before-and-after balance summary in the Deposit Output section.

UI Element	Type	What It Does	Notes / Validation
Customer Name	Dropdown	Select the customer making the deposit.	<i>Required. Populated from AppState.customers.</i>
Account	Dropdown	Select the account to deposit into from the selected customer's account list.	<i>Required. Populated from the selected customer's accountList.</i>
Deposit Amount	Text field	Enter the dollar amount to deposit.	<i>Required. Must be greater than \$0. Calls Account.deposit(amount).</i>
Previous Balance	Display field	Shows the account balance before the deposit is applied.	<i>Auto-populated after deposit. Read-only.</i>
Deposit Amount (output)	Display field	Shows the deposit amount that was applied.	<i>Auto-populated after deposit. Read-only.</i>

UI Element	Type	What It Does	Notes / Validation
New Balance	Display field	Shows the updated account balance after the deposit.	<i>Auto-populated after deposit. Read-only.</i>
Deposit	Button	Executes Account.deposit(amount) and refreshes the Deposit Output section with the before and after balances.	<i>Prints confirmation to stdout. Updates display fields on screen.</i>
Return to Teller Screen	Button	Returns to the Teller home screen.	<i>No data is lost — the deposit is already committed when Deposit was clicked.</i>

3.5 Withdraw from Account

Create New Customer

Create New Customer

Customer ID

Address

City

State

Zipcode

First Name

Last Name

Create Customer

Return to Teller Screen

Ready.

After creation, return to the Teller Screen.

Figure 3.5 – Withdraw from Account

This screen allows a Teller to process a cash withdrawal from a customer's account. After a successful withdrawal, the screen shows a before-and-after balance summary in the Withdrawal Output section.

UI Element	Type	What It Does	Notes / Validation
Customer Name	Dropdown	Select the customer requesting the withdrawal.	<i>Required. Populated from AppState.customers.</i>
Account	Dropdown	Select the account to withdraw from.	<i>Required. Populated from the selected customer's accountList. CDAccount withdrawals must use earlyWithdraw() instead.</i>
Withdrawal Amount	Text field	Enter the dollar amount to withdraw.	<i>Required. Must be greater than \$0. Overdraft transfer fires automatically if a savings account is linked.</i>
Previous Balance	Display field	Shows the account balance before the withdrawal is applied.	<i>Auto-populated after withdrawal. Read-only.</i>
Withdrawal Amount (output)	Display field	Shows the withdrawal amount that was applied.	<i>Auto-populated after withdrawal. Read-only.</i>
New Balance	Display field	Shows the updated account balance after the withdrawal.	<i>Auto-populated after withdrawal. Read-only.</i>
Withdraw	Button	Executes Account.withdraw(amount). If balance goes negative and an overdraft account is linked, handleOverdraft() transfers the deficit from savings at no fee. If no backup, a \$25 overdraft fee is charged.	<i>Calls Account.withdraw(). Overdraft logic in CheckingsAccount.handleOverdraft().</i>
Return to Teller Screen	Button	Returns to the Teller home screen.	<i>No data is lost — the withdrawal is already committed when Withdraw was clicked.</i>

3.6 Stop Payment

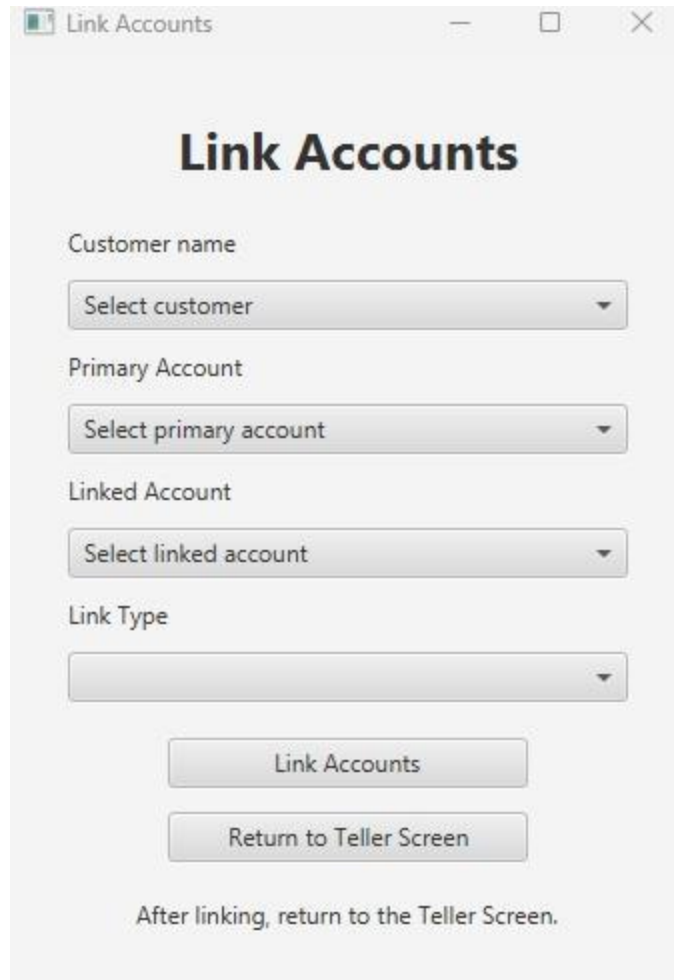


Figure 3.6 – Stop Payment

This screen allows a Teller to place a stop-payment order on a specific check. The system charges a service fee and flags the check number so that it is rejected if presented for payment.

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the Customer ID of the account holder requesting the stop payment.	<i>Required.</i>
Checking Account Number	Text field	Enter the checking account number associated with the check to be stopped.	<i>Required. Must be a valid TMBAccount or GDAccount for the customer.</i>
Check Number	Text field	Enter the number of the check to be stopped, as printed on the check.	<i>Required. Calls CheckingsAccount.stopPayment(checkNumber). Currently a stub — full list implementation is planned.</i>
Service Fee	Display field	Read-only field showing the stop-payment service fee. This	<i>Fixed fee. Not editable. Per UML Sequence Diagram (Scenario D): \$25.</i>

UI Element	Type	What It Does	Notes / Validation
		amount is automatically deducted from the checking account.	
Apply Stop Payment	Button	Records the stop-payment order, deducts the service fee from the checking account, and flags the check number.	<i>Calls <code>CheckingsAccount.stopPayment(checkNum)</code>. Fee deducted at submission.</i>
Return to Teller Screen	Button	Cancels the stop-payment request and returns to the Teller home screen.	<i>No data is saved and no fee is charged.</i>

3.7 Review Customer Accounts

Progress Time

Progress System Time

Current Date: 2026-08-26

Enter number of days

Progress Time

Return to System Controller

Figure 3.7 – Review Customer Accounts

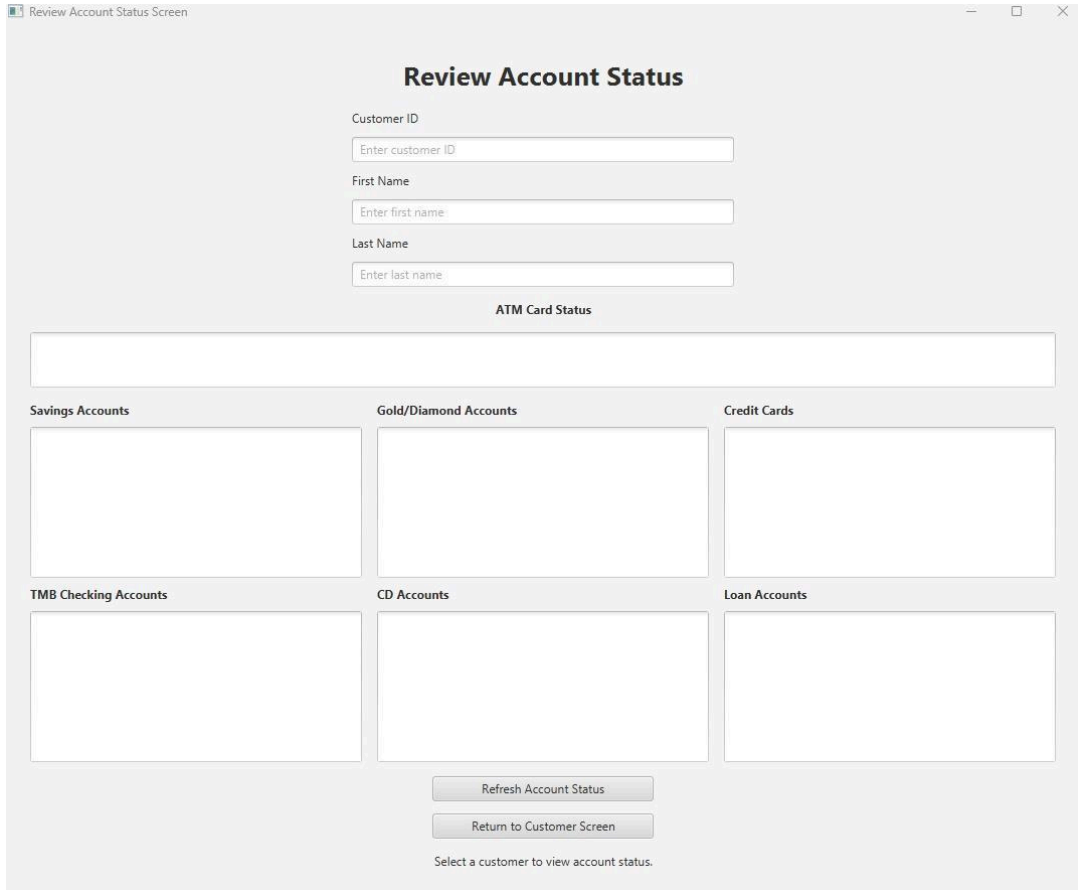
This screen allows a Teller to look up all accounts associated with a specific customer and view a detailed summary including account number, type, balance, recent debit activity, status, and any notes.

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the Customer ID of the customer whose accounts you want to review.	<i>Required. Must match an existing customer.</i>
Search Accounts	Button	Queries the system for all accounts in the customer's accountList and populates the summary table.	<i>Table is cleared and repopulated on each search.</i>
Return to Teller Screen	Button	Returns to the Teller home screen without saving any changes.	<i>This screen is read-only; no data is modified.</i>

UI Element	Type	What It Does	Notes / Validation
Account Number (column)	Table column	Displays the UUID account number for each account found.	<i>Read-only.</i>
Type (column)	Table column	Displays the account type: Savings, CD, TMB Checking, GD Checking, or Credit Card.	<i>Read-only.</i>
Balance (column)	Table column	Displays getBalance() for each account.	<i>Read-only. Formatted as currency.</i>
Recent Debit (column)	Table column	Shows the most recent debit transaction amount posted to each account.	<i>Read-only. Useful for spotting unusual activity.</i>
Status (column)	Table column	Displays account status: Active, Frozen, Overdue, Closed, etc. CreditCard shows isProblemAccount.	<i>Read-only.</i>
Notes (column)	Table column	Displays flags such as overdraft backup status or problem-account flags.	<i>Read-only. overdraftBackup = true appears here for linked savings accounts.</i>

4. Bank Manager Screens

Bank Managers have access to five administrative screens. These screens are accessed from the Manager home screen shown below and require Manager-level credentials.



Review Account Status

Customer ID
Enter customer ID

First Name
Enter first name

Last Name
Enter last name

ATM Card Status

Savings Accounts

Gold/Diamond Accounts

Credit Cards

TMB Checking Accounts

CD Accounts

Loan Accounts

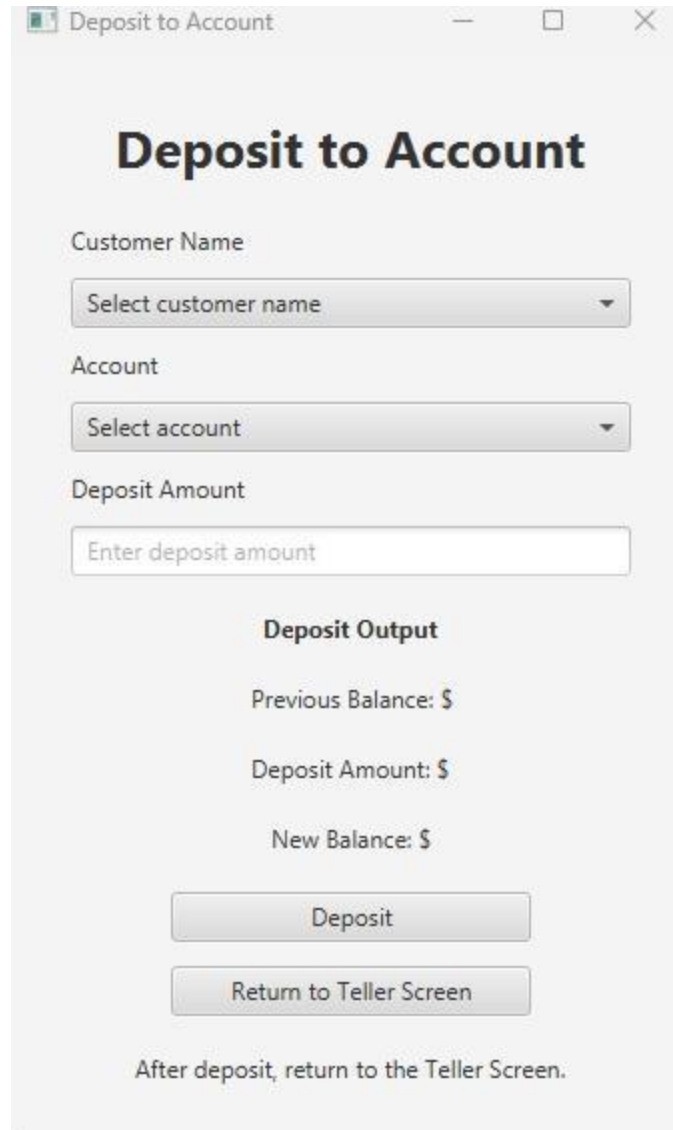
Refresh Account Status

Return to Customer Screen

Select a customer to view account status.

Manager Screen – Home

4.1 Create New Loan



Deposit to Account

Deposit to Account

Customer Name

Select customer name

Account

Select account

Deposit Amount

Enter deposit amount

Deposit Output

Previous Balance: \$

Deposit Amount: \$

New Balance: \$

Deposit

Return to Teller Screen

After deposit, return to the Teller Screen.

Figure 4.1 – Create New Loan

This is a new screen that allows a Bank Manager to create a new loan for an existing customer. The customer is selected from a dropdown, and the loan type, principal, interest rate, and due date grace period are entered to set up the loan.

UI Element	Type	What It Does	Notes / Validation
Customer Name	Dropdo wn	Select the customer for whom the loan is being created.	<i>Required. Populated from AppState.customers.</i>
Loan Type	Dropdo wn	Select the loan product: Mortgage Loan (15 or 30-year terms) or Short-Term Loan (5-year term).	<i>Required. Determines the TermBasedLoan subclass instantiated and the amortisation schedule.</i>

UI Element	Type	What It Does	Notes / Validation
Principal / Loan Amount	Text field	Enter the total loan amount being issued to the customer.	<i>Required. Stored as principal in the Loan object. Used in calculateLoanRepayment().</i>
Interest Rate	Text field	Enter the annual interest rate as a percentage (e.g. 6.5 for 6.5%).	<i>Required. Stored as interest_rate. Used in TermBasedLoan.calculateLoanRepayment() amortisation formula.</i>
Due Date Grace Period	Text field	Enter the number of days after the due date before a payment is considered late. Defaults to 10.	<i>Required. Stored as dueDateInterval. Affects applyLateFee() and flagIfMissed() threshold.</i>
Create Loan	Button	Creates the loan with the specified parameters, calculates the monthly payment via calculateLoanRepayment(), and links it to the customer record. Returns to the Manager Screen on success.	<i>Returns to the Manager Screen on success.</i>
Return to Manager Screen	Button	Cancels loan creation and returns to the Manager home screen.	<i>No data is saved.</i>

4.2 Set Interest Rates on Loans

Figure 4.2 – Set Interest Rates on Loans

This screen allows a Bank Manager to update the interest rate applied to loan products. The new rate takes effect on the date specified.

UI Element	Type	What It Does	Notes / Validation
Loan Type	Dropdown	Select the loan product to update: Mortgage Loan (15 or 30-year terms) or Short-Term Loan (5-year term).	<i>Required. Rate is applied only to the selected loan type.</i>
Interest Rate	Text field	Enter the new annual interest rate as a percentage (e.g. enter 6.5 for 6.5%). Stored as interest_rate in the Loan object.	<i>Required. Must be a positive number. Used in <code>TermBasedLoan.calculateLoanRepayment()</code> amortisation formula.</i>

UI Element	Type	What It Does	Notes / Validation
Effective Date	Text field	Enter the date on which the new rate becomes active (MM/DD/YYYY). Loans renewing on or after this date will use the new rate.	<i>Required. Cannot be a past date.</i>
Update Loan Rate	Button	Saves the new interest rate for the selected loan type and effective date. Returns to the Manager Screen after confirmation.	<i>Rate change is logged. Existing loans recalculate via <code>calculateLoanRepayment()</code> on their next billing cycle.</i>
Return to Manager Screen	Button	Cancels the rate update and returns to the Manager home screen without saving.	<i>No changes are committed.</i>

4.3 Set Interest Rates on CDs / Checking

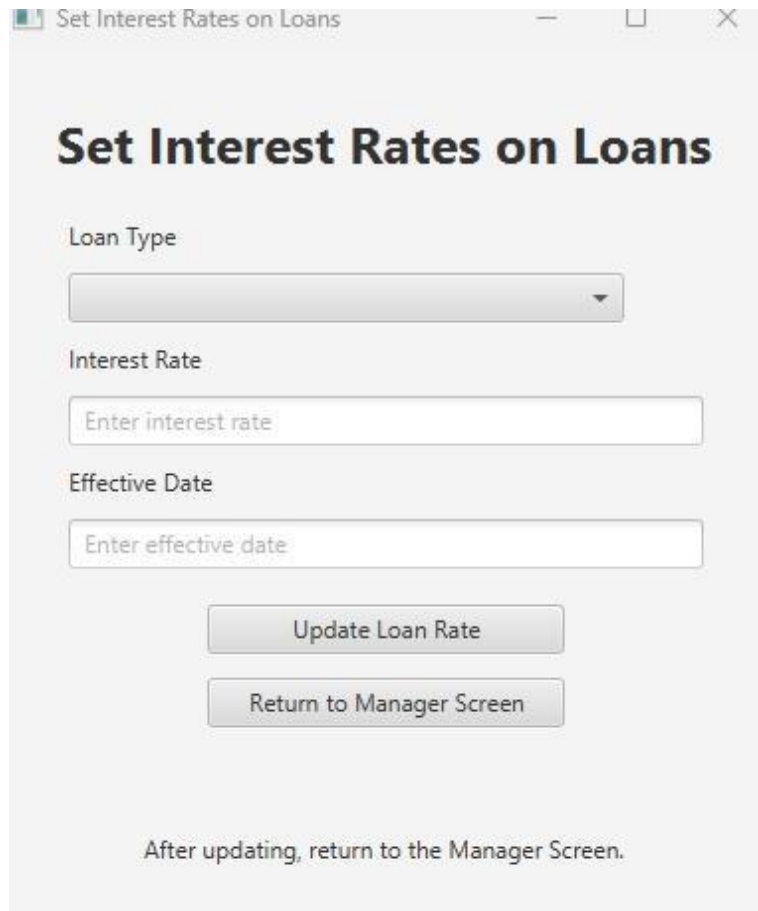
Figure 4.3 – Set Interest Rates on CDs / Checking

This screen allows a Bank Manager to set the interest rate for deposit account types: CD, TMB Checking, Gold Diamond Checking, and Savings.

UI Element	Type	What It Does	Notes / Validation
Account Type	Dropdown	Select the deposit account type to update: CD Account, TMB Checking, Gold Diamond (GD) Checking, or Savings.	<i>Required. Rate applies only to the selected account type.</i>

UI Element	Type	What It Does	Notes / Validation
Interest Rate	Text field	Enter the new interest rate as a decimal (e.g. 0.035 for 3.5%). For GDAccount this is the daily flexible rate — applyInterest() sets interestRate = 0.5 x interestAccount.getBalance().	<i>Required. Must be a positive number. Stored as interestRate on SavingsAccount.</i>
Effective Date	Text field	Enter the date the new rate takes effect (MM/DD/YYYY).	<i>Required. Cannot be a past date.</i>
Update Account Rate	Button	Commits the new rate for the selected account type. Returns to the Manager Screen on success.	<i>Daily interest on Savings calls SavingsAccount.applyDailyInterest(). CD rate is locked at creation via CDAccount.fixedRate.</i>
Return to Manager Screen	Button	Cancels the update and returns to the Manager home screen without saving.	<i>No changes are committed.</i>

4.4 Establish CDs



Set Interest Rates on Loans

Loan Type

Interest Rate

Effective Date

Update Loan Rate

Return to Manager Screen

After updating, return to the Manager Screen.

Figure 4.4 – Establish CDs

This screen allows a Bank Manager to create a new Certificate of Deposit account for an existing customer. The customer is selected from a dropdown. The Manager sets the opening deposit, fixed rate, CD term, and early-withdrawal penalty. Note: the Early Withdrawal Penalty defaults to 50.0.

UI Element	Type	What It Does	Notes / Validation
Customer Name	Dropdown	Select the customer for whom the CD is being established.	<i>Required. Populated from <code>AppState.customers</code>.</i>
CD Term	Dropdown	Select the term length for the CD.	<i>Required. Determines the maturity date. Stored as <code>CDAccount.maturityDate (java.util.Date)</code>.</i>
Initial Deposit	Text field	Enter the opening deposit amount for the CD. Deposited via <code>Account.deposit()</code> at creation.	<i>Required. Must be greater than \$0.</i>
Interest Rate	Text field	Enter the annual fixed rate to apply to this CD (as a decimal, e.g. 0.04 for 4%).	<i>Required. Stored as <code>CDAccount.fixedRate</code> — locked for the life of the term.</i>
Early Withdrawal Penalty	Text field	Enter the penalty amount charged if the customer withdraws before maturity. Defaults to 50.0.	<i>Required. Stored as <code>CDAccount.earlyPenalty</code>. Applied by <code>CDAccount.earlyWithdraw()</code> (stub — full logic planned).</i>
Establish CD	Button	Creates the <code>CDAccount</code> with the specified parameters and links it to the customer record.	<i>Maturity date auto-calculated from today plus selected term. Standard <code>withdraw()</code> is blocked on CDs — use <code>earlyWithdraw()</code> instead.</i>
Return to Manager Screen	Button	Cancels CD creation and returns to the Manager home screen.	<i>No data is saved.</i>

4.5 Process Checks



Set Interest Rates on CDs / Checking

Set Interest Rates on CDs / Checking

Account Type

Interest Rate

Effective Date

Update Account Rate

Return to Manager Screen

After updating, return to the Manager Screen.

Figure 4.5 – Process Checks

This screen allows a Bank Manager to process a check presented at the branch. The system validates the check, applies overdraft fee-waiver logic, and records the result.

UI Element	Type	What It Does	Notes / Validation
Customer ID	Text field	Enter the Customer ID of the account holder whose check is being processed.	<i>Required. Must match an existing customer.</i>
Checking Account Number	Text field	Enter the checking account number from which the check will be drawn.	<i>Required. Must be a valid TMBAccount or GDAccount for the customer.</i>
Check Number	Text field	Enter the check number as printed on the physical check.	<i>Required. System cross-references against the stopPayments list before processing.</i>
Check Amount	Text field	Enter the dollar value written on the check.	<i>Required. Must be greater than \$0.</i>
Process Check	Button	Processes the check: (1) verifies not on stop-payment list, (2) calls requestFeeWaiver(checkAmt) — returns true if (balance - checkAmt) >= \$5,000, (3) if waived: honors check; if not waived and balance	<i>Fee waiver is a local balance check in this build. Full CheckpointManager.hasSufficientFunds() integration is planned.</i>

UI Element	Type	What It Does	Notes / Validation
		insufficient: returns check with \$25 overdraft fee via handleOverdraft().	
Return to Manager Screen	Button	Cancels check processing and returns to the Manager home screen.	<i>No data is saved.</i>

5. System Controller Screens

The System Controller role manages the technical health of the banking application. These screens are used during system startup, pre-production validation, and recovery operations. They are not accessible to Customers, Tellers, or Bank Managers.

5.1 Load Basic Data



Figure 5.1 – Load Basic Data

This screen is used at system startup to load the core banking data files into the application. The screen shows a 'Data Source File' field pre-filled with 'src/data.csv'. A status label shows 'Ready.' when the screen opens and updates after each action.

UI Element	Type	What It Does	Notes / Validation
Data Source File	Text field	File path to the customer and account data CSV file. Pre-filled with 'src/data.csv'. This combined CSV contains customer records plus pipe-delimited account data on the same row.	<i>Required. Must be a valid, readable CSV path. Parsed by <code>CsvManager.fetchCustsAndAccountsFromCSV()</code>.</i>
Load Data	Button	Reads the CSV at the specified path into <code>AppState.customers</code> (an <code>ArrayListManager<Customer></code> sorted by last name). All accounts are also parsed and linked to their customers. Controllers access this data via <code>AppState.customers</code> .	<i>Calls <code>CsvManager.fetchCustsAndAccountsFromCSV()</code> followed by <code>CsvManager.handleOverdrafts()</code> to restore overdraft links. Status label updates on completion.</i>

UI Element	Type	What It Does	Notes / Validation
Validate Data	Button	Runs a validation pass on the data file to confirm format integrity without committing it.	<i>Run Validate before Load to catch CSV format errors early.</i>
Return to System Controller	Button	Returns to the System Controller home screen without loading data.	<i>No data is loaded if this button is clicked without first pressing Load Data.</i>

5.2 Run Simple Regression Tests

Figure 5.2 – Run Simple Regression Tests

This screen allows the System Controller to run a configurable set of regression tests to verify that core system functions are working correctly before go-live or after a restore. The tests correspond to the JUnit 5 suites in the tests/ directory.

UI Element	Type	What It Does	Notes / Validation
Account Tests (checkbox)	Check box	When checked, runs AccountTests.java: deposit, withdraw, overdraft protection, fund transfer, overdraft penalty, and savings interest tests.	<i>7 tests. Checked by default. Tests TMBAccount and SavingsAccount behaviour.</i>

UI Element	Type	What It Does	Notes / Validation
Loan Tests (checkbox)	Check box	When checked, runs LoanTests.java: loan creation, payment, repayment schedule, full amortisation, problem flag, and credit card charge/bill tests.	<i>12 tests. Checked by default. Tests MortgageLoan, ShortTermLoan, and CreditCard.</i>
File / Data Tests (checkbox)	Check box	When checked, runs MainTests.java: verifies that src/data.csv is present and readable.	<i>1 test. Checked by default.</i>
Run Selected Tests	Button	Executes all checked test suites in sequence using JUnit 5. A results summary shows pass/fail for each test.	<i>Tests run against sandboxed data; no live customer data is modified.</i>
View Test Results	Button	Displays the results of the most recently completed test run, including any failure messages.	<i>Results are available until the next test run overwrites them.</i>
Return to System Controller	Button	Returns to the System Controller home screen.	

5.3 Check Point System

Create Check Screen

Customer Name

Select customer name ▼

Sending account

Select account ▼

Check Amount

Enter check amount:

Receivers name

Select customer name ▼

Create Check

Return to Customer Screen

After submitting, return to the Customer Screen.

Figure 5.3 – Check Point System

This screen manages the system checkpoint and disaster-recovery process. The UI has been simplified to four buttons. CheckpointManager is now implemented — Begin Check Point writes all customer and account data to data.csv via CsvManager.

UI Element	Type	What It Does	Notes / Validation
Begin Check Point	Button	Initiates the checkpoint process. Calls CheckpointManager.EXEC_CHECK_POINT(AppState.customers), which calls CsvManager.writeCustomersToCsv() to persist all customer and account data to src/data.csv, then calls CsvManager.handleOverdrafts() to re-verify overdraft links.	<i>CheckpointManager is now implemented. Run this before shutting down to preserve all changes.</i>

UI Element	Type	What It Does	Notes / Validation
Shut Down System	Button	Gracefully shuts down the banking application.	<i>WARNING: Always run Begin Check Point before shutting down to avoid data loss since the last checkpoint.</i>
Restore Saved State	Button	Reloads the system state from the last saved data.csv checkpoint by calling Load Basic Data.	<i>Restores customer and account data from the last checkpoint file.</i>
Return to System Controller	Button	Returns to the System Controller home screen without performing any checkpoint or shutdown action.	

6. Quick Reference

6.1 Screen Directory

Screen Name	Role	Primary Action
Credit Card Screen	Customer	Make payment toward credit card balance
Review Account Status	Customer	View all personal account balances and statuses
Input Check Screen	Customer	Submit a check; overdraft waiver logic applied via requestFeeWaiver()
Create New Customer	Teller	Register a new customer (field order: ID, Address, City, State, Zipcode, First, Last)
Create New Account	Teller	Open a new account; customer selected by name dropdown, account type dropdown
Link Accounts	Teller	Link savings to checking for overdraft protection; all dropdowns
Deposit to Account	Teller	Deposit funds into a customer account; shows before/after balance
Withdraw from Account	Teller	Process a counter cash withdrawal; shows before/after balance
Stop Payment	Teller	Place a stop-payment order on a check
Review Customer Accounts	Teller	Search and view a customer's full account summary
Create New Loan	Bank Manager	Create a mortgage or short-term loan for a customer
Set Interest Rates on Loans	Bank Manager	Update interest rate for Mortgage or Short-Term Loan
Set Interest Rates on CDs/Checking	Bank Manager	Update interest rate for deposit account types
Establish CDs	Bank Manager	Create a new CD account; customer selected by dropdown; penalty defaults to 50.0
Process Checks	Bank Manager	Process a branch check with fee-waiver logic
Load Basic Data	System Controller	Load combined customer+account CSV via CsvManager at startup
Run Simple Regression Tests	System Controller	Run AccountTests, LoanTests, and MainTests suites
Check Point System	System Controller	Save checkpoint via CheckpointManager, shut down, or restore state

6.2 Fee Schedule

Fee Type	Amount & Trigger
Overdraft Fee	\$25.00 – charged via <code>handleOverdraft()</code> when checking balance goes negative and no savings backup is linked.
Stop Payment Fee	Per UML Sequence Diagram (Scenario D): \$25.00 charged at time of stop-payment submission.
TMB Transfer Fee	\$0.75 per transfer (<code>tsFee</code>). Charged by <code>TMBAccount.transfer()</code> before executing the transfer.
TMB Monthly Transfer Fee	\$1.25 per monthly transfer (<code>monthlyTransferFee</code>). Charged by <code>TMBAccount.makeMonthlyTransfer()</code> .
GD Below-Minimum Transfer Fee	\$0.75 per transfer when <code>GDAccount</code> balance is below \$5,000. Charged by <code>GDAccount.chargeBelowMinFee()</code> .
GD Below-Minimum Monthly Fee	\$1.25 per monthly transfer when <code>GDAccount</code> balance is below \$5,000.
CD Early Withdrawal Penalty	Varies – set at CD creation as <code>CDAccount.earlyPenalty</code> . Applied by <code>earlyWithdraw()</code> (stub; full logic planned).
Late Loan Payment Fee	\$75.00 (<code>lateFee</code> in <code>TermBasedLoan</code>) – applied by <code>applyLateFee()</code> when payment is past <code>paymentDueDate</code> .
Overdraft Fee Waiver	\$25 fee is waived if <code>requestFeeWaiver(checkAmt)</code> returns true — i.e. $(\text{balance} - \text{checkAmt}) \geq \$5,000$.

6.3 Revision History

Ver.	Date	Description	Author
1.0	April 5, 2025	Initial release with all 16 screens from the CSC 406 app.	Aidan Godair
1.1	April 2026	Updated to reflect <code>prototypeBranch</code> implementation: 7-field Customer, UUID account numbers, accurate fee values, new account types (<code>GDAccount</code> , <code>CDAccount</code> , <code>CreditCard</code> , loan hierarchy), <code>requestFeeWaiver()</code> logic, and test suite details.	Aidan Godair
1.2	April/May 2026	Updated all changed screens with new JavaFX UI screenshots. Added new screens: Deposit to Account (Teller) and Create New Loan (Manager). Updated Create New Customer, Create New Account, Link Accounts, Withdraw from Account, Establish CDs, Load Basic Data, and Check Point System. Teller and Manager home screens added. Screen inventory and Quick Reference updated to reflect 18 total screens.	Aidan Godair
1.3	May 2026	Rebuilt manual as a formatted Word document (.docx) with all 24 updated JavaFX UI screenshots from <code>Bank_Screens.zip</code> replacing the prior PDF-embedded images. Converted from PDF to .docx format, preserving all content, UI element tables, business rules, and section structure. Cover page, heading styles, and table formatting refreshed. Version and title block updated on cover page.	Aidan Godair